

# Derek Seber

## Senior Software Engineer

Phoenixville, PA 19460 | djseber3@gmail.com | (215) 853-6927 | LinkedIn | GitHub

## SKILLS

---

### Professional Programming Language

- Python
- Go (Golang)
- JavaScript / TypeScript
- Java
- C
- SQL / NoSQL
- Bash

### MLOps / Machine Learning Engineering

- Machine Learning System Design & Production Deployment
- Real-time Data & Feature Engineering Pipelines (Kafka, Flink)
- Model Serving & Inference at Scale (Triton, AWS SageMaker, ONNX Runtime)
- Distributed Systems & Cloud Infrastructure (AWS, Kubernetes, Docker)
- Deep Learning & LLMs (PyTorch, Transformers)
- Data Engineering for ML Systems (Python, SQL, S3)
- MLOps CI/CD & Model Evaluation Pipelines
- GPU-Accelerated ML Systems (TensorRT, AWS G5)

## EDUCATION

---

University of Maryland (UMD) College Park

2013 – 2017

Bachelor of Science, Computer Science

## PROFESSIONAL EXPERIENCE

---

Plaid

01/2026 – Present

### MLOps Engineer

- **Architected** heterogeneous Graph Neural Network (GNN) models using **Python** and **PyTorch Geometric** to map multi-entity transaction networks for the Plaid Protect Platform, deploying custom **GraphSAGE** architectures on **AWS SageMaker** to significantly reduce global platform fraud rates and minimize financial losses.
- **Engineered** a real-time feature engineering pipeline using **Apache Flink** and **Apache Kafka** to ingest and process massive scales of dynamic network attributes into **Feast** and **Redis**, achieving ultra-low-latency feature retrieval to stop multi-platform scams instantly.
- **Optimized** high-throughput model serving infrastructure by deploying Triton Inference Server with custom **C++** backends across **AWS EKS** clusters, driving real-time inference for the Trust Index 3 engine to scale transaction processing volume efficiently.
- **Integrated** domain-specific financial transformer models using **Hugging Face Transformers** and **Pandas** to clean and vectorize highly fragmented merchant names, eliminating data noise in the fraud graph to minimize user onboarding friction.

- **Developed** automated evaluation and hard negative mining frameworks using **PyTorch**, **NumPy**, and **Scikit-learn** to continuously train models against adaptive fraud patterns, maintaining strict **KYC/AML** compliance without degrading conversion rates.

## Plaid

08/2025 – 12/2025

### Senior Software Engineer

- **Built** a high-speed bank data enrichment system using **Go (Golang)**, **Python**, and **FastAPI** by writing optimized concurrency patterns with **Goroutines**, **Channels**, and **Sync.Pool** object reuse to ensure instant data delivery for users during peak traffic hours.
- **Cut** external AI cloud fees by setting up an intelligent database memory layer using **Redis Enterprise** and **Milvus Vector Database** to save and reuse past answers instead of paying for duplicate **OpenAI API** processing.
- **Sped up** automated AI data analysis on **AWS EC2 G5 GPU** servers by shrinking model sizes via **TensorRT INT8** quantization and hosting them on **NVIDIA Triton Inference Server** to reduce compute infrastructure overhead.
- **Protected** client applications from system crashes during external AI outages by building an automated safety switch in **Go** using **go-resiliency/circuitbreaker** tools and **RE2 regex** matching to keep the core service continuously active.
- **Improved** app responsiveness under heavy transaction spikes by setting up an asynchronous task delegation network using **Celery**, **Gunicorn**, and **RabbitMQ** to handle heavy data validation and tokenization behind the scenes.
- **Blocked** millions of dollars in fraudulent bank transfers by co-developing a real-time risk alert streaming system using **Go**, **confluent-kafka-go**, and **Apache Kafka** hosted on **AWS MSK** to analyze user bank history before money settled.
- **Saved** network bandwidth and data transfer costs across internal systems by shifting microservice communications from legacy **REST/JSON** formats to high-performance **gRPC** and **Protocol Buffers (Protobuf v3)** data structures.
- **Guaranteed** high-speed risk assessments by using **ONNX Runtime Go bindings** to run Python-trained **XGBoost** and **LightGBM** business models directly inside the fast **Go** core application runtimes.
- **Lowered** cloud operational overhead by deploying the system onto **Amazon EKS (Kubernetes)** using **Docker** containers and **Terraform** scripts, causing servers to scale up automatically based on **Prometheus** and **Grafana** traffic monitors.
- Collaborated closely with cross functional teams in a high velocity **Scrum** environment.

## Turing

03/2025 – 07/2025

### AI Training Engineer

- Engineered high-fidelity training pipelines using **Python** and **SQL** to extract and clean complex architectural datasets, enabling the model to learn multi-tier **AWS infrastructure patterns** and **enterprise PostgreSQL schema** designs.
- Debugged and refined hundreds of high-priority **C++ training prompts**, ensuring the LLM correctly implemented **STL** containers and followed modern **C++20** standards for backend efficiency.
- Implemented a custom **code-verification sandbox** using **Go**, **Docker**, and **Bash scripts** to automatically execute, unit-test, and benchmark LLM-generated code, ensuring 100% logic validation before data ingestion.
- Executed advanced **RLHF** (Reinforcement Learning from Human Feedback) workflows within **Labelbox**, ranking model responses based on **Go** concurrency safety (race condition detection) and **Python PEP-8** idiomatic standards.
- Developed automated **Bash** and **Python** utility scripts to manage high-volume data uploads via **Labelbox APIs**, streamlining the expert-review process and increasing training data throughput by 50%.
- Designed "Chain-of-Thought" (**CoT**) **prompting** strategies to train models on **AWS CloudFormation** and **Terraform** logic, teaching the AI to reason through security group configurations and **VPC** subnetting.
- Orchestrated large-scale data processing tasks on **AWS EC2** and **S3**, utilizing **Docker** containers to ensure environment parity between local development and the **cloud-based AI training cluster**.

- Leveraged **AWS cloud-native tools** and **Docker** orchestration to build a robust **CI/CD pipeline** for rapid model evaluation and deployment of backend-focused **AI agents**.

## EPAM Systems

08/2024 – 03/2025

### Founding Engineer

- **Built** a full-stack sales intelligence MVP using **Next.js**, **TypeScript**, **FastAPI**, and **Python** to automate background profile discovery and turn raw contact data into personalized event ideas.
- **Architected** an asynchronous data enrichment and event generation pipeline using **Celery** and **Redis** to run heavy AI workloads behind the scenes without blocking user interface interactions.
- **Engineered** a multi-step data ingestion system using **FastAPI** and **Pydantic** to handle CSV file uploads, automatic column mapping, data validation, and database deduplication constraints.
- **Implemented** a public data discovery workflow combining **SerpAPI** and the **OpenAI API** to scrape web content and extract structured career history, education, and interest signals.
- **Enforced** strict data reliability by using **OpenAI Structured Outputs** with **Pydantic** models to attach verifiable confidence scores and source metadata references to every extracted profile trait.
- **Designed** a custom scoring and ranking algorithm inside **PostgreSQL** that evaluates location alignment, signal confidence, and executive seniority to filter recommendations into clear high, medium, and low priority tiers.
- **Integrated** generative AI prompts to draft dual-tone client invitation messages and scheduled automated executive email summaries delivered via **SendGrid** using **Celery** cron schedules.
- **Developed** a high-performance web dashboard using **React**, **Next.js**, and **TypeScript**, constructing interactive analytics grids with **Tailwind CSS** to display enriched profiles, signal confidence breakdowns, and event recommendations cleanly.
- **Minimized** frontend data fetching latency and state synchronization overhead by implementing **TanStack Query (React Query)** and **Axios** to cache API data and manage server state updates smoothly without redundant web page reloads.
- **Built** a cross-platform mobile application using **React Native**, **TypeScript**, and **Expo** to deliver real-time sales insights, enabling sales leaders to review AI-generated event ideas and client profiles directly from mobile devices.
- **Optimized** mobile app performance and data freshness by implementing a localized client-side caching layer using **MMKV** storage and configuring push alerts through **Firebase Cloud Messaging (FCM)**.

## CVS Health

12/2017 – 07/2024

### Software Engineer

- **Built** a high-concurrency, **HIPAA-compliant** national vaccine booking portal using **React**, **TypeScript**, **Node.js**, and **Express**, creating smooth step-by-step patient forms that scaled effortlessly during massive traffic surges to secure high booking completion rates.
- **Integrated** third-party healthcare systems including **EHR platforms** and **Oracle Health APIs** to enable real-time patient record synchronization and streamline clinical workflow interoperability across provider networks.
- **Accelerated** patient checkout speeds and form submission performance by optimizing client-side state transitions with **Redux Toolkit** and introducing strict request validation rules via **Zod**.
- **Engineered** a real-time mail-order prescription tracking dashboard using **Python**, **Django REST Framework (DRF)**, and **React**, providing patients with transparent status updates and significantly lowering customer support call center workloads.
- **Decoupled** slower legacy mainframe lookups from the user interface by building an asynchronous data fetching pipeline using **Celery** and **Redis** to refresh pharmacy order data safely behind the scenes.
- **Ensured** platform stability and code quality before deployment by writing comprehensive automated testing suites with **Pytest** for backend logic, alongside **Jest** and **React Testing Library** for frontend interactive views.
- **Protected** patient data privacy by managing **HIPAA-regulated Protected Health Information (PHI)** using **JSON Web Tokens (JWT)**, **HTTPS** data protocols, and application-layer data encryption algorithms.